

— the integration interval has to be partitioned by several fitting points with the solution being matched at each such point. For more details see [1].

CITED REFERENCES AND FURTHER READING:

- Acton, F.S. 1970, *Numerical Methods That Work*; 1990, corrected edition (Washington, DC: Mathematical Association of America).
- Keller, H.B. 1968, *Numerical Methods for Two-Point Boundary-Value Problems*; reprinted 1991 (New York: Dover).
- Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), §7.3.5 – §7.3.6.[1]

18.3 Relaxation Methods

In *relaxation methods* we replace ODEs by approximate *finite difference equations* (FDEs) on a grid or mesh of points that spans the domain of interest. As a typical example, we could replace a general first-order differential equation

$$\frac{dy}{dx} = g(x, y) \quad (18.3.1)$$

with an algebraic equation relating function values at two points $k, k - 1$:

$$y_k - y_{k-1} - (x_k - x_{k-1}) g \left[\frac{1}{2}(x_k + x_{k-1}), \frac{1}{2}(y_k + y_{k-1}) \right] = 0 \quad (18.3.2)$$

The form of the FDE in (18.3.2) illustrates the idea, but not uniquely: There are many ways to turn the ODE into an FDE. When the problem involves N coupled first-order ODEs represented by FDEs on a mesh of M points, a solution consists of values for N dependent functions given at each of the M mesh points, or $N \times M$ variables in all. The relaxation method determines the solution by starting with a guess and improving it, iteratively. As the iterations improve the solution, the result is said to *relax* to the true solution.

While several iteration schemes are possible, for most problems our old standby, multi-dimensional Newton's method, works well. The method produces a matrix equation that must be solved, but the matrix takes a special, "block diagonal" form that allows it to be inverted far more economically both in time and storage than would be possible for a general matrix of size $(MN) \times (MN)$. Since MN can easily be several thousand or more, this is crucial for the feasibility of the method.

Our implementation couples at most pairs of points, as in equation (18.3.2). More points can be coupled, but then the method becomes more complex. We will provide enough background so that you can write a more general scheme if you have the patience to do so.

Let us develop a general set of algebraic equations that represent the ODEs by FDEs. The ODE problem is exactly identical to that expressed in equations (18.0.1) – (18.0.3), where we had N coupled first-order equations that satisfy n_1 boundary conditions at one end of the interval and $n_2 = N - n_1$ boundary conditions at the other. We first define a mesh or grid by a set of $k = 0, 1, \dots, M - 1$ points at which we supply values for the independent variable x_k . In particular, x_0 is the initial boundary and x_{M-1} is the final boundary. We use the notation y_k to refer to the entire set of dependent variables $y_0, y_1, \dots, y_{N-1}$ at point x_k . At an arbitrary point k in the middle of the mesh, we approximate the set of N first-order ODEs by algebraic relations of the form

$$0 = \mathbf{E}_k \equiv \mathbf{y}_k - \mathbf{y}_{k-1} - (x_k - x_{k-1}) \mathbf{g}_k(x_k, x_{k-1}, \mathbf{y}_k, \mathbf{y}_{k-1}), \quad k = 1, 2, \dots, M - 1 \quad (18.3.3)$$

The notation signifies that $\mathbf{g}_k$ can be evaluated using information from both points $k, k - 1$. The FDEs labeled by $\mathbf{E}_k$ provide N equations coupling $2N$ variables at points $k, k - 1$. There

are $M - 1$ points, $k = 1, 2, \dots, M - 1$, at which difference equations of the form (18.3.3) apply. Thus the FDEs provide a total of $(M - 1)N$ equations for the MN unknowns. The remaining N equations come from the boundary conditions.

At the first boundary we have

$$0 = \mathbf{E}_0 \equiv \mathbf{B}(x_0, y_0) \quad (18.3.4)$$

while at the second boundary

$$0 = \mathbf{E}_M \equiv \mathbf{C}(x_{M-1}, y_{M-1}) \quad (18.3.5)$$

The vectors $\mathbf{E}_0$ and $\mathbf{B}$ have only n_1 nonzero components, corresponding to the n_1 boundary conditions at x_0 . It will turn out to be useful to take these nonzero components to be the *last* n_1 components. In other words, $E_{j,0} \neq 0$ only for $j = n_2, n_2 + 1, \dots, N - 1$. At the other boundary, only the first n_2 components of $\mathbf{E}_M$ and $\mathbf{C}$ are nonzero: $E_{j,M} \neq 0$ only for $j = 0, 1, \dots, n_2 - 1$.

The “solution” of the FDE problem in (18.3.3) – (18.3.5) consists of a set of variables $y_{j,k}$, the values of the N variables y_j at the M points x_k . The algorithm we describe below requires an initial guess for the $y_{j,k}$. We then determine increments $\Delta y_{j,k}$ such that $y_{j,k} + \Delta y_{j,k}$ is an improved approximation to the solution.

Equations for the increments are developed by expanding the FDEs in first-order Taylor series with respect to small changes $\Delta \mathbf{y}_k$. At an interior point, $k = 1, 2, \dots, M - 1$, this gives

$$\mathbf{E}_k(\mathbf{y}_k + \Delta \mathbf{y}_k, \mathbf{y}_{k-1} + \Delta \mathbf{y}_{k-1}) \approx \mathbf{E}_k(\mathbf{y}_k, \mathbf{y}_{k-1}) + \sum_{n=0}^{N-1} \frac{\partial \mathbf{E}_k}{\partial y_{n,k-1}} \Delta y_{n,k-1} + \sum_{n=0}^{N-1} \frac{\partial \mathbf{E}_k}{\partial y_{n,k}} \Delta y_{n,k} \quad (18.3.6)$$

For a solution we want the updated value $\mathbf{E}(\mathbf{y} + \Delta \mathbf{y})$ to be zero, so the general set of equations at an interior point can be written in matrix form as

$$\sum_{n=0}^{N-1} S_{j,n} \Delta y_{n,k-1} + \sum_{n=N}^{2N-1} S_{j,n} \Delta y_{n-N,k} = -E_{j,k}, \quad j = 0, 1, \dots, N - 1 \quad (18.3.7)$$

where

$$S_{j,n} = \frac{\partial E_{j,k}}{\partial y_{n,k-1}}, \quad S_{j,n+N} = \frac{\partial E_{j,k}}{\partial y_{n,k}}, \quad n = 0, 1, \dots, N - 1 \quad (18.3.8)$$

The quantity $S_{j,n}$ is an $N \times 2N$ matrix at each point k . Each interior point thus supplies a block of N equations coupling $2N$ corrections to the solution variables at the points $k, k - 1$.

Similarly, the algebraic relations at the boundaries can be expanded in a first-order Taylor series for increments that improve the solution. Since $\mathbf{E}_0$ depends only on $\mathbf{y}_0$, we find at the first boundary

$$\sum_{n=0}^{N-1} S_{j,n} \Delta y_{n,0} = -E_{j,0}, \quad j = n_2, n_2 + 1, \dots, N - 1 \quad (18.3.9)$$

where

$$S_{j,n} = \frac{\partial E_{j,0}}{\partial y_{n,0}}, \quad n = 0, 1, \dots, N - 1 \quad (18.3.10)$$

At the second boundary,

$$\sum_{n=0}^{N-1} S_{j,n} \Delta y_{n,M-1} = -E_{j,M}, \quad j = 0, 1, \dots, n_2 - 1 \quad (18.3.11)$$

where

$$S_{j,n} = \frac{\partial E_{j,M}}{\partial y_{n,M-1}}, \quad n = 0, 1, \dots, N - 1 \quad (18.3.12)$$

X X X X X		V	B		
X X X X X		V	B		
X X X X X		V	B		
X X X X X X X X X X		V	B		
X X X X X X X X X X		V	B		
X X X X X X X X X X		V	B		
X X X X X X X X X X		V	B		
X X X X X X X X X X		V	B		
	X X X X X X X X X X	V	B		
	X X X X X X X X X X	V	B		
	X X X X X X X X X X	V	B		
	X X X X X X X X X X	V	B		
	X X X X X X X X X X	V	B		
		X X X X X X X X X X	V	B	
		X X X X X X X X X X	V	B	
		X X X X X X X X X X	V	B	
		X X X X X X X X X X	V	B	
			X X X X X	V	B
			X X X X X	V	B

Figure 18.3.1. Matrix structure of a set of linear finite difference equations (FDEs) with boundary conditions imposed at both endpoints. Here X represents a coefficient of the FDEs, V represents a component of the unknown solution vector, and B is a component of the known right-hand side. Empty spaces represent zeros. The matrix equation is to be solved by a special form of Gaussian elimination. (See text for details.)

We thus have in equations (18.3.7) – (18.3.12) a set of linear equations to be solved for the corrections Δy , iterating until the corrections are sufficiently small. The equations have a special structure, because each $S_{j,n}$ couples only points $k, k - 1$. Figure 18.3.1 illustrates the typical structure of the complete matrix equation for the case of five variables and four mesh points, with three boundary conditions at the first boundary and two at the second. The 3×5 block of nonzero entries in the top left-hand corner of the matrix comes from the boundary condition $S_{j,n}$ at point $k = 0$. The next three 5×10 blocks are the $S_{j,n}$ at the interior points, coupling variables at mesh points (2,1), (3,2), and (4,3). Finally we have the block corresponding to the second boundary condition.

We can solve equations (18.3.7) – (18.3.12) for the increments Δy using a form of Gaussian elimination that exploits the special structure of the matrix to minimize the total number of operations, and that minimizes storage of matrix coefficients by packing the elements in a special blocked structure. (You might wish to review Chapter 2, especially §2.2, if you are unfamiliar with the steps involved in Gaussian elimination.) Recall that Gaussian elimination consists of manipulating the equations by elementary operations such as dividing rows of coefficients by a common factor to produce unity in diagonal elements, and adding appropriate multiples of other rows to produce zeros below the diagonal. Here we take advantage of the block structure by performing a bit more reduction than in pure Gaussian elimination, so that the storage of coefficients is minimized. Figure 18.3.2 shows the form that we wish to achieve by elimination, just prior to the backsubstitution step. Only a small subset of the reduced $MN \times MN$ matrix elements needs to be stored as the elimination progresses. Once the matrix elements reach the stage in Figure 18.3.2, the solution follows quickly by a backsubstitution procedure.

Furthermore, the entire procedure, except the backsubstitution step, operates only on one block of the matrix at a time. The procedure contains four types of operations: (1) partial reduction to zero of certain elements of a block using results from a previous step; (2) elimination of the square structure of the remaining block elements such that the square section contains unity along the diagonal, and zero in off-diagonal elements; (3) storage of the

(a)	1	0	0	S	S											V	S
	0	1	0	S	S											V	S
	0	0	1	S	S											V	S
	Z	Z	Z	D	D	D	D	D	A	A						V	A
	Z	Z	Z	D	D	D	D	D	A	A						V	A
	Z	Z	Z	D	D	D	D	D	A	A						V	A
	Z	Z	Z	D	D	D	D	D	A	A						V	A
	Z	Z	Z	D	D	D	D	D	A	A						V	A
	Z	Z	Z	D	D	D	D	D	A	A						V	A
	Z	Z	Z	D	D	D	D	D	A	A						V	A
(b)	1	0	0	S	S											V	S
	0	1	0	S	S											V	S
	0	0	1	S	S											V	S
	0	0	0	1	0	0	0	0	S	S						V	S
	0	0	0	0	1	0	0	0	S	S						V	S
	0	0	0	0	0	1	0	0	S	S						V	S
	0	0	0	0	0	0	1	0	S	S						V	S
	0	0	0	0	0	0	0	1	S	S						V	S
	0	0	0	0	0	0	0	0	1	S	S					V	S
	0	0	0	0	0	0	0	0	0	1	S	S				V	S

Figure 18.3.4. Reduction process for intermediate blocks of the matrix in Figure 18.3.1. (a) Original form, (b) final form. (See text for explanation.)

the matrix. This means that the n_1 boundary conditions at the first point must contain some dependence on the first $j=0, 1, \dots, n_1 - 1$ dependent variables, $y[j][0]$. If not, then the original square $n_1 \times n_1$ subsection of the first block will appear to be singular, and the method will fail. Alternatively, we would have to allow the search for pivot elements to involve all N columns of the block, and this would require column swapping and far more bookkeeping. The code provides a simple method of reordering the variables, i.e., the columns of the s matrix, so that this can be done easily. End of important detail.

Next consider the block of N equations representing the FDEs that describe the relation between the $2N$ corrections at points 1 and 0. The elements of that block, together with results from the previous step, are illustrated in Figure 18.3.4. Note that by adding suitable multiples of rows from the first block we can reduce to zero the first n_1 columns of the block (labeled by “Z”), and, to do so, we will need to alter only the columns from n_1 to $N - 1$ and the vector element on the right-hand side. Of the remaining columns we can diagonalize a square subsection of $N \times N$ elements, labeled by “D” in the figure. In the process we alter the final set of n_2 columns, denoted “A” in the figure. The second half of the figure shows the block when we finish operating on it, with the stored $n_2 \times N$ elements labeled by “S.”

If we operate on the next set of equations corresponding to the FDEs coupling corrections at points 2 and 1, we see that the state of available results and new equations exactly reproduces the situation described in the previous paragraph. Thus, we can carry out those steps again for each block in turn through block $M - 1$. Finally on block M we encounter the remaining boundary conditions.

Figure 18.3.5 shows the final block of n_2 FDEs relating the N corrections for variables at mesh point $M - 1$, together with the result of reducing the previous block. Again, we can first use the prior results to zero the first n_1 columns of the block. Now, when we diagonalize the remaining square section, we strike gold: We get values for the final n_2 corrections at mesh point $M - 1$.

With the final block reduced, the matrix has the desired form shown previously in Figure 18.3.2, and the matrix is ripe for backsubstitution. Starting with the bottom row and working up toward the top, at each stage we can simply determine one unknown correction in terms of known quantities.

The object `Solve` organizes the steps described above. The principal procedures used in the algorithm are performed by functions called internally by `Solve`. The function `red` eliminates leading columns of the s matrix using results from prior blocks. `pinvs` diagonalizes the square subsection of s and stores unreduced coefficients. `bksub` carries out the

(a)	0	0	0	1	0	0	0	0	S	S	V	S
	0	0	0	0	1	0	0	0	S	S	V	S
	0	0	0	0	0	1	0	0	S	S	V	S
	0	0	0	0	0	0	1	0	S	S	V	S
	0	0	0	0	0	0	0	1	S	S	V	S
					Z	Z	Z	D	D		V	A
(b)					Z	Z	Z	D	D		V	A
	0	0	0	1	0	0	0	0	S	S	V	S
	0	0	0	0	1	0	0	0	S	S	V	S
	0	0	0	0	0	1	0	0	S	S	V	S
	0	0	0	0	0	0	1	0	S	S	V	S
					0	0	0	1	0		V	S
					0	0	0	0	1		V	S

Figure 18.3.5. Reduction process for the last (lower-right) block of the matrix in Figure 18.3.1. (a) Original form, (b) final form. (See text for explanation.)

backsubstitution step. The user of `Solve` must understand the calling arguments, as described below, and supply an object `Difeq`, called by `Solve`, with a method `smatrix` that evaluates the s matrix for each block.

Most of the arguments in the constructor call to `Solve` have already been described, but some require discussion. On input, array $y[j][k]$ contains the initial guess for the solution, with j labeling the dependent variables at mesh points k . The problem involves ne FDEs spanning points $k=0, \dots, m-1$. nb boundary conditions apply at the first point $k=0$. The array `indexv[j]` establishes the correspondence between columns of the s matrix; equations (18.3.8), (18.3.10), and (18.3.12); and the dependent variables. As described above, it is essential that the nb boundary conditions at $k=0$ involve the dependent variables referenced by the first nb columns of the s matrix. Thus, columns j of the s matrix can be ordered by the user in `Difeq` to refer to derivatives with respect to the dependent variable `indexv[j]`.

The function only attempts `itmax` correction cycles before returning, even if the solution has not converged. The parameters `conv`, `slowc`, and `scalv` relate to convergence. Each inversion of the matrix produces corrections for ne variables at m mesh points. We want these to become vanishingly small as the iterations proceed, but we must define a measure for the size of corrections. This error “norm” is very problem-specific, so the user might wish to rewrite this section of the code as appropriate. In the program below we compute a value for the average correction `err` by summing the absolute value of all corrections, weighted by a scale factor appropriate to each type of variable:

$$\text{err} = \frac{1}{m \times ne} \sum_{k=0}^{m-1} \sum_{j=0}^{ne-1} \frac{|\Delta Y[j][k]|}{\text{scalv}[j]} \quad (18.3.13)$$

When `err` $\leq$ `conv`, the method has converged. Note that the user gets to supply an array `scalv` that measures the typical size of each variable.

Obviously, if `err` is large, we are far from a solution, and perhaps it is a bad idea to believe that the corrections generated from a first-order Taylor series are accurate. The number `slowc` modulates the application of corrections. After each iteration we apply only a fraction of the corrections found by matrix inversion:

$$Y[j][k] \rightarrow Y[j][k] + \frac{\text{slowc}}{\max(\text{slowc}, \text{err})} \Delta Y[j][k] \quad (18.3.14)$$

Thus, when `err` $>$ `slowc` only a fraction of the corrections are used, but when `err` $\leq$ `slowc` the entire correction gets applied.

As already mentioned, the constructor initializes the array `y[0..ne-1][0..m-1]` in `Solve` with the trial solution. Internally, workspace arrays `c[0..ne-1][0..ne-nb][0..m]`, `s[0..ne-1][0..2*ne]` are allocated. The array `c` is the blockbuster: It stores the unreduced elements of the matrix built up for the backsubstitution step. If there are `m` mesh points, then there will be `m+1` blocks, each requiring `ne` rows and `ne-nb+1` columns. Although large, this is small compared with $(ne \times m)^2$ elements required for the whole matrix if we did not break it into blocks.

We now describe the workings of the user-supplied object `Difeq`. The constructor can be used to pass problem-specific information from your main program. The object must contain a method `smatrix` with the following declaration:

```
void smatrix(const Int k, const Int k1, const Int k2, const Int jsf,
            const Int isf, const Int isf, VecInt_I &indexv, MatDoub_0 &s,
            MatDoub_I &y);
```

As the declaration shows, the only information passed from `Difeq` to `Solve` is the matrix of derivatives `s[0..ne-1][0..2*ne]`; all other arguments are input to `smatrix` and should not be altered. `k` indicates the current mesh point, or block number. `k1`, `k2` label the first and last points in the mesh. If `k = k1` or `k > k2`, the block involves the boundary conditions at the first or final point; otherwise the block acts on FDEs coupling variables at points `k-1`, `k`.

The convention on storing information into the array `s[i][j]` follows that used in equations (18.3.8), (18.3.10), and (18.3.12): Rows `i` label equations and columns `j` refer to derivatives with respect to dependent variables in the solution. Recall that each equation will depend on the `ne` dependent variables at either one or two points. Thus, `j` runs from 0 to either `ne-1` or `2*ne-1`. The column ordering for dependent variables at each point must agree with the list supplied in `indexv[j]`. Thus, for a block not at a boundary, the first column multiplies $\Delta Y(l=indexv[0], k-1)$, and the column `ne` multiplies $\Delta Y(l=indexv[0], k)$. The parameters `is1`, `isf` give the numbers of the starting and final rows that need to be filled in the `s` matrix for this block. `jsf` labels the column in which the difference equations $E_{j,k}$ of equations (18.3.3)–(18.3.5) are stored. Thus, $-s[i][jsf]$ is the vector on the right-hand side of the matrix. The reason for the minus sign is that `smatrix` supplies the actual difference equation, $E_{j,k}$, not its negative. Note that `Solve` supplies a value for `jsf` such that the difference equation is put in the column *just after* all derivatives in the `s` matrix. Thus, `smatrix` expects to find values entered into `s[i][j]` for rows `is1 ≤ i ≤ isf` and `0 ≤ j ≤ jsf`.

Finally, the quantities `s[0..nsi-1][0..nsj-1]` and `y[0..nyj-1][0..nyk-1]` supply `smatrix` with storage for `s` and the values of the solution variables `y` for this iteration. An example of how to use this routine is given in the next section.

Detailed implementing code for `Solve` is given in a Webnote [1], many ideas in which are due to Eggleton [2].

18.3.1 “Algebraically Difficult” Sets of Differential Equations

Relaxation methods allow you to take advantage of an additional opportunity that, while not obvious, can speed up some calculations enormously. It is not necessary that the set of variables $y_{j,k}$ correspond exactly with the dependent variables of the original differential equations. They can be related to those variables through algebraic equations. Obviously, it is necessary only that the solution variables allow us to *evaluate* the functions y , g , $\mathbf{B}$, $\mathbf{C}$ that are used to construct the FDEs from the ODEs. In some problems, g depends on functions of y that are known only implicitly, so that iterative solutions are necessary to evaluate functions in the ODEs. Often one can dispense with this “internal” nonlinear problem by defining a new set of variables from which both y , g and the boundary conditions can be obtained directly. A typical example occurs in physical problems where the equations require the solution of a complex equation of state that can be expressed in more convenient terms using variables other than the original dependent variables in the ODE. While this approach is analogous to performing an *analytic* change of variables directly on the original ODEs, such an analytic transformation might be prohibitively complicated. The change of variables in the relaxation method is easy and requires no analytic manipulations.